Hardware Sudoku Solver with AC-3 Constraint Propagation, MRV-Guided Backtracking, and VGA Display on Basys3 FPGA

Karney Jayanath (Sr. No. 26831), Department of Electronic Systems Engineering
Indian Institute of Science, Bengaluru – 560 012

Abstract

This paper presents the complete architectural design, Verilog HDL implementation, and hardware demonstration of a real-time Sudoku constraint-satisfaction solver on the Digilent Basys3 (Artix-7 XC7A35T) FPGA. The system embodies a three-layer hardware pipeline: (i) a* parallel AC-3 arc-consistency engine *that processes all 81 candidate domains simultaneously in a single-pass architecture, converging to a fixed-point in $O(P)$ clock cycles where P is the number of AC-3 passes required; (ii) a* Minimum Remaining Values (MRV) handoff FSM *that extracts AC-3-solved singletons, orders the residual unsolved cells by ascending candidate count without explicit sorting hardware, and loads a LIFO shift-register stack; and (iii) a* chronological backtracking engine *consisting of interlocked* `control`, `updater`, *and* `checker` *FSMs that operate at 100 MHz with a single-cycle conflict check latency.

Puzzle ingestion is supported over a 115 200-baud UART interface from a host PC or interactively via five directional buttons and 16 slide switches on the board. A two-stage pipelined VGA renderer drives a 640×480@60 Hz display with four navigable tabs: FPGA Solve, User Mode, Statistics, and Reset. A six-counter performance monitor captures cycle-accurate metrics—total solve cycles, AC-3 phase cycles, pass count, AC-3-resolved cell count, backtrack event count, and peak stack depth—and transmits the results to the host as a structured 93-byte response frame after each solve. The design meets timing closure at 100 MHz on all paths, including 729-bit wide candidate-store combinational buses relaxed via multicycle-path constraints, and is verified through end-to-end hardware demonstration including the Arto Inkala puzzle, widely considered one of the hardest 9×9 Sudoku instances.

I Introduction

Sudoku is a well-known binary constraint satisfaction problem (CSP) with 9^{81} possible assignments, reduced in practice to a tractable search space by arc-consistency preprocessing and variable-ordering heuristics [5, 4]. Unlike software solvers that exploit dynamic memory and branch-prediction hardware, an FPGA implementation must express the same algorithmic structure as a fixed datapath. The challenge is therefore not algorithmic novelty but *architectural mapping*: translating AC-3 propagation, MRV ordering, and backtracking into synthesisable Verilog that meets timing at 100 MHz on a resource-constrained Artix-7 device.

Design Goals and Constraints

The project targets the following hard constraints on the Basys3 board:

1. **Single clock domain at 100 MHz.** A pixel-enable signal derived by clock division provides the 25 MHz VGA reference without a second clock domain, eliminating all cross-domain hazards.
2. **No external memory.** All state—the 81×4-bit display RAM, the 81×9-bit candidate store, and the 81-entry backtracking stack—must fit in distributed RAM or flip-flops within the FPGA fabric.
3. **Timing closure at 100 MHz.** Wide combinational buses (729 bits for the candidate store; 324 bits for the board-flat bus) require multicycle-path constraints to route without hold-time repair buffers.
4. **Cycle-accurate profiling.** Six hardware counters provide ground truth for comparing algorithmic variants across puzzles.

Contributions

The principal hardware contributions of this work are:

- A **parallel single-pass AC-3 engine** in which all 81 arc-processing units fire simultaneously, achieving $O(P)$ convergence in clock cycles rather than the $O(|\mathcal{A}| \cdot D^2)$ complexity of sequential AC-3 software implementations [3].
- A **shift-register MRV sorter** that achieves correct MRV ordering by exploiting the LIFO property of the updater stack, requiring no comparators, no explicit sort network, and no additional registers beyond the stack itself.

- A **pipelined conflict detector** that avoids integer division in index computation by replacing runtime arithmetic with registered lookup tables, enabling a 27-comparison full-board scan at 100 MHz within a 30-cycle pipeline.
- A **two-stage VGA pipeline** that decouples geometry decode from colour assignment, ensuring no single combinational path from pixel coordinates to RGB output exceeds one clock period at 100 MHz.

The remainder of this report is organised as follows. Section II presents the top-level architecture and memory organisation. Sections III–XI describe each subsystem in detail. Section XV reports observed hardware behaviour, and Section XVI discusses design tradeoffs and lessons learned.

II System Architecture

2.1 Top-Level Overview

The top-level module `top_basys3` integrates all subsystems under a 100 MHz system clock. Reset is synchronised through a two-flip-flop synchroniser driven by the centre button (BTNC). The principal data path is a 4-bit cell-value bus (`cell_data[3:0]`) and a 7-bit address bus (`cell_addr[6:0]`) that fan out simultaneously to the UART receive path, the display RAM, the candidate store, and the Sudoku solver. Figure 1 summarises the top-level connectivity.

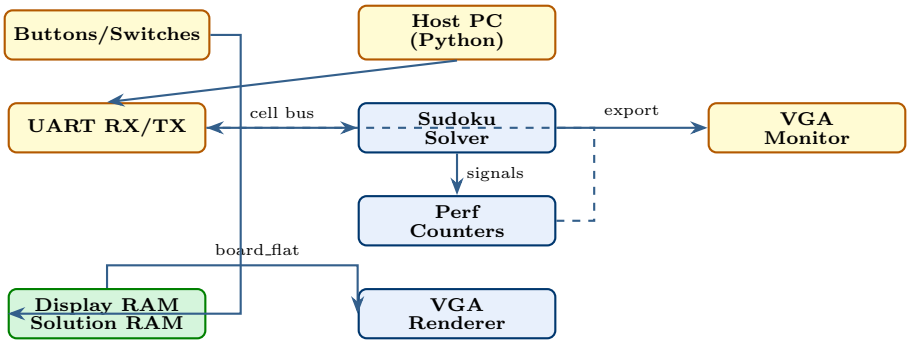

Figure 1: Simplified top-level architecture of the Basys3 Sudoku Solver. Dashed arrow: performance bytes transmitted back to host after solution.

2.2 Memory Organisation

Three independent register arrays are maintained in the top module. The *display RAM* (`display_ram[0:80]`, 81×4 bits) holds the currently shown board, updated by UART load, user switch entry, solver writebacks, and reveal/hint operations. The *solution RAM* (`solution_ram[0:80]`, 81×4 bits) accumulates solver-exported values and is read non-destructively for hints. An 81-bit *given mask* (`given_mask`) marks cells that were loaded as puzzle clues so that they cannot be overwritten by user input or hint injection. A further 81-bit *error mask* (`error_mask`) is produced by the conflict-detection pipeline and forwarded to the renderer.

2.3 Clock and Pixel Enable

A divide-by-four counter (`clk_div`) derives a 25 MHz pixel-enable signal (`clk_25`) from the 100 MHz system clock. All VGA-related logic is clocked at 100 MHz but gated by `clk_25`, ensuring pixel-synchronous operation without a second clock domain and eliminating clock-domain crossing hazards.

III UART Interface

3.1 Receiver and Transmitter

Both UART modules are parameterised by `CLKS_PER_BIT` = 868, corresponding to 115 200 baud at 100 MHz (100,000,000/115,200 ≈ 868). The receiver uses a 10-bit counter to accommodate this value and includes a two-flip-flop synchroniser initialised to logic-high (idle state) on reset. Start-bit detection samples at $\lfloor \text{CLKS\_PER\_BIT}/2 \rfloor - 3$ rather than the exact midpoint, compensating for the one-cycle detection latency and improving eye-centre accuracy. Framing errors (stop bit sampled low) cause the received byte to be silently discarded and the FSM to return to Idle, preventing corrupted data from entering the solver pipeline. The transmitter drives the standard NRZ 8N1 format and asserts `busy` for the duration of each transmission; the top FSM polls `busy` before queuing the next byte.

3.2 Protocol

The host Python script transmits exactly 81 bytes, each carrying a BCD digit (0 for empty, 1–9 for given clues) in the lower nibble. The FPGA echoes the solution as 81 bytes followed by 11 performance bytes and a sentinel `0xFE`, for a total of 93 bytes per transaction. The first byte is buffered on arrival in Idle and replayed internally once the Recv state starts processing, avoiding off-by-one addressing.

IV Candidate Store and AC-3 Engine

4.1 Candidate Store

The `candidate_store` module maintains a flat register array `masks_reg[0:80]` of 9-bit one-hot candidate masks. Bit k (0-indexed) being set means digit $k{+}1$ is still a valid candidate for that cell. On puzzle load the top module drives `cs_ext_wr_en` with the appropriate mask: a given digit d maps to the singleton mask `9'b1` shifted left by $d{-}1$; an empty cell receives the all-ones mask `9'b111111111`. The `rd_mask_all` output bus ($729 = 81 \times 9$ bits) exposes the entire array combinatorially to both the AC-3 controller and the handoff FSM.

4.2 AC-3 Arc-Consistency Algorithm

Arc consistency enforces the constraint that for every cell c_i and every value v in its domain, there exists at least one value v' in each peer cell c_j (sharing a row, column, or 3×3 box) that is compatible with v. For Sudoku this simplifies to: if a peer cell c_j has been assigned a unique value v', then v' must be eliminated from the domain of c_i.

The `ac3_controller` implements this with a parallel array of 81 arc-processing units, each reading the $9 \times 9 = 81$ peers of its assigned cell and computing the union of all singleton peer masks. Bits present in this union are eliminated from the local candidate mask in a single clock cycle. The controller iterates passes until no mask changes (fixed-point convergence) or a domain wipes out to zero (contradiction, signalled via `contra`). The `solving` output remains asserted throughout; the `ac3_pass_done` pulse marks the start of each new pass.

4.3 AC-3 Start Trigger

AC-3 starts exactly once per puzzle. The signal `ac3_start` is derived as a one-cycle rising-edge pulse on the address-81 sentinel: `ac3_start = input_finish && !input_finish_prev`, where `input_finish` is simply `(in_addr == 7'd81)`. This ensures that the candidate store is fully populated before the AC-3 engine begins.

V Handoff FSM and MRV Ordering

5.1 Role and Motivation

The handoff FSM bridges the AC-3 engine and the backtracking solver. It serves three purposes:

(i) **Singleton injection.** Cells reduced to a singleton domain by AC-3 are deterministically solved; they should be written directly into the checker array as confirmed values rather than queued for backtracking. This avoids wasting backtracking cycles on trivially determined cells.

(ii) **Empty-cell registration.** The remaining unsolved cells must be enumerated and their board indices registered in the updater's shift-register stack before backtracking begins.

(iii) **MRV ordering.** The unsolved cells are registered in order of *decreasing* candidate count (most-candidates first), which, combined with the LIFO property of the shift-register stack, places the most-constrained cells (fewest remaining candidates) at the *front* of the processing queue—the correct MRV ordering without any explicit sorting logic.

5.2 State Encoding and Detailed Transitions

The handoff FSM uses an 8-state, 4-bit one-hot-compatible binary encoding. Table 1 provides a precise description of each state.

Table 1: Handoff FSM State Reference

Enc.	State	Description and Exit Condition
4'd0	HO_Idle	Waits for either `ac3_contra` (go to HO_Contra) or `ac3_done` pulse (go to HO_Scan, reset `ac3_solved[]` array).
4'd1	HO_Contra	Terminal error state. `out_answer` is forced to `2'b01` (NO SOLUTION). Self-loops forever; cannot be exited without a global reset.
4'd2	HO_Scan	Iterates `ho_cell` from 0 to 80. For each cell: reads the 9-bit mask from `cs_rd_mask_all`; skips if `given_track[ho_cell]` is set; tests singleton-ness using $m \neq 0 \wedge (m\&(m-1)) = 0$; if singleton, asserts `ho_valid`, drives `ho_addr`/`ho_data`, and sets `ac3_solved[ho_cell]`. Advances `ho_cell` every cycle; exits to HO_Finish when `ho_cell = 80`.
4'd3	HO_Finish	Emits a single-cycle sentinel pulse: `ho_valid=1`, `ho_addr=81`, `ho_data=0`. This locks the AC-3-solved cell count in the updater's `empty_num_r`. Proceeds unconditionally to HO_Wait.
4'd4	HO_Wait	Single-cycle initialisation: sets `mrv_pass` $\leftarrow 9$, `ho_cell` $\leftarrow 0$. Proceeds to MRV_Sort.
4'd5	MRV_Sort	Outer loop: `mrv_pass` counts down from 9 to 2. Inner loop: `ho_cell` scans 0 to 80. For each cell that is (a) not a given, (b) not AC-3-solved, and (c) has popcount = `mrv_pass`: asserts `ho_valid` and emits `ho_addr`/`ho_data=0`. When `ho_cell=80`: if `mrv_pass=2` go to MRV_Finish; else decrement `mrv_pass`, reset `ho_cell=0`, continue.
4'd6	MRV_Finish	Emits terminal sentinel (`ho_valid=1`, `ho_addr=81`). Transitions to HO_Done (*not* HO_Contra—see bug fix below).
4'd7	HO_Done	Neutral do-nothing terminal state. `ctrl_out_answer` propagates normally. Introduced in rev-6 to fix the permanent NO-SOLUTION regression.

The state transition diagram is shown in Figure 2.

5.3 Singleton Detection

The singleton test used in HO_Scan exploits a classical bit-manipulation identity: a positive integer m is a power of two (i.e., has exactly one bit set) if and only if $m \neq 0$ and $m \,\&\, (m-1) = 0$. Applied to the 9-bit candidate mask:

$$\text{is\_singleton}(m) \iff (m \neq 9'b0) \wedge \big(m \,\&\, (m - 9'b1) = 9'b0\big)$$

This requires no popcount hardware: a single 9-bit AND and a 9-bit comparator, both synthesising to a handful of LUTs. The corresponding digit is recovered by the `mask_to_digit` priority encoder (lowest set bit index +1).

5.4 MRV Ordering via the LIFO Stack Trick

The MRV heuristic requires that the most-constrained cell (fewest remaining candidates) be attempted first by the backtracker. A naïve hardware implementation would require a sorting network of $O(n \log n)$ comparators, which is expensive in LUT area. The present design avoids this entirely by exploiting two facts:

(1) The `updater`'s position register is a *shift-register stack*: each newly received cell is prepended (shift-left), so the last cell loaded ends up at index 0, the first position to be processed.

(2) The MRV_Sort loop emits cells in *decreasing* popcount order (pass 9 down to pass 2). The last batch emitted (pass 2, fewest candidates) is prepended last, landing at the front of the stack.

Consequently, when the backtracker pops cells from index 0 upward, it naturally encounters the most-constrained cells first—correct MRV ordering achieved with zero sorting hardware and only a 4-bit down-counter for `mrv_pass`. Table 2 illustrates this with a small example.

Table 2: Illustrative MRV Ordering via LIFO Prepend. Stack index 0 is processed first.

Emission order	Cell addr	Popcount (pass)	Final stack index
1st	45	5	3 (processed 4th)
2nd	12	4	2 (processed 3rd)
3rd	67	3	1 (processed 2nd)
4th	30	2	0 (processed 1st)

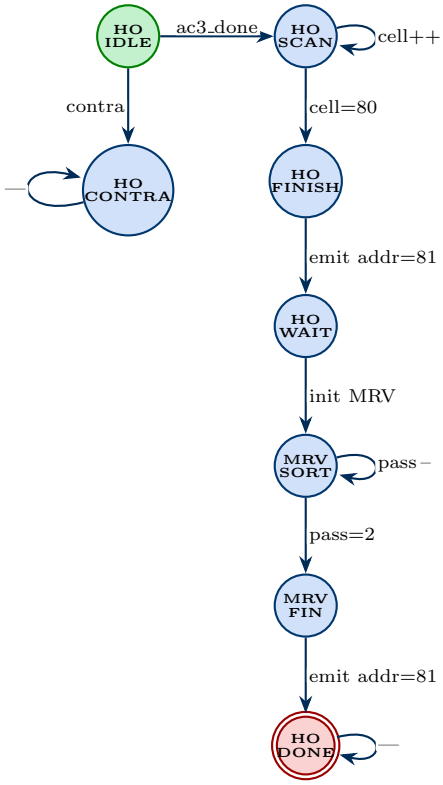

Figure 2: Handoff FSM state transition diagram. HO_Contra is the terminal error state; HO_Done is the neutral terminal state introduced in rev-6 to prevent `out_answer` from being permanently forced to NO SOLUTION.

5.5 AC-3 Cell Injection (HO_Scan)

In HO_Scan the FSM iterates over all 81 cells. For each cell it reads the 9-bit mask from `cs_rd_mask_all`, checks the `given_track` register (to skip pre-filled givens), and tests singleton-ness using the power-of-two identity:

$$\text{is\_solved}(m) \iff (m \neq 0) \ \wedge \ \big(m \ \&\ (m-1) = 0\big).$$

If the cell is solved and not a given, the FSM emits a write pulse (`ho_valid`, `ho_addr`, `ho_data`) to the `control/updater` pipeline and marks the cell in `ac3_solved[]`. The digit is recovered from the mask by `mask_to_digit`, a priority encoder returning the index of the lowest set bit plus one. A registered copy of this write pulse feeds `cell_solved_ac3` to the performance counter.

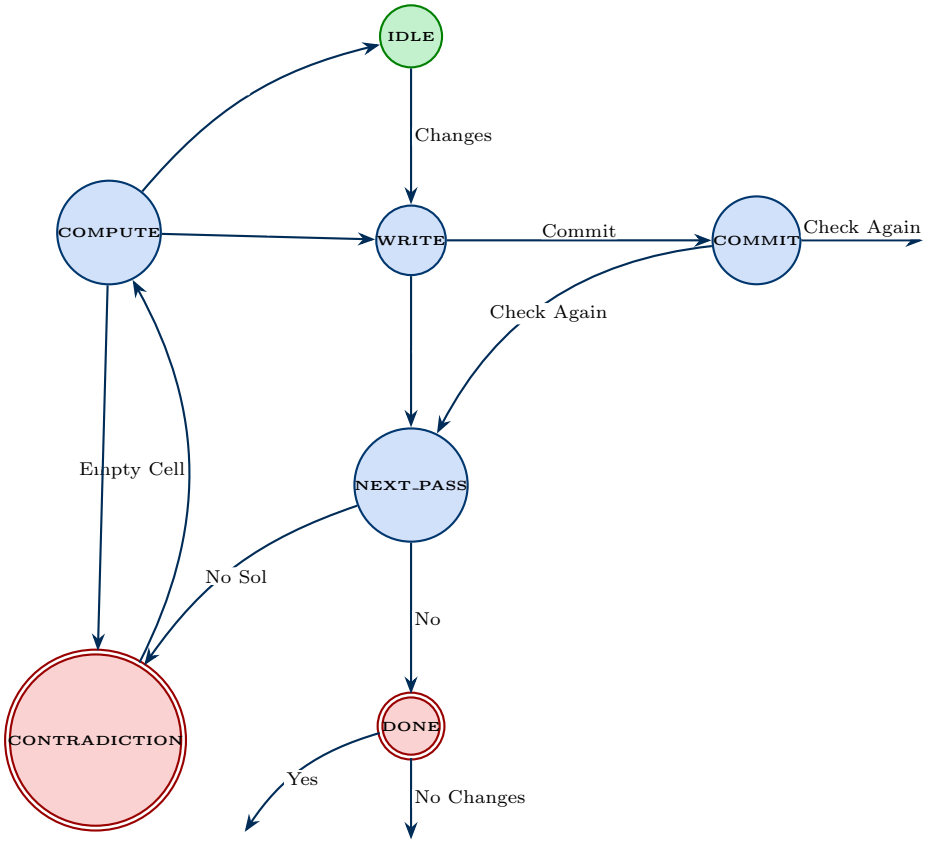

Figure 3: AC-3 solver finite state machine (FSM). The system starts from **IDLE**, computes constraints in **COMPUT**, updates candidate values in **WRITE**, saves updates in **COMMIT**, and checks if another pass is needed in **NEXT_PASS**. If no candidates remain, the solver enters **CONTRADICTION**; otherwise, successful completion leads to **DONE**.

5.6 MRV Ordering (MRV_Sort)

After the sentinel (address 81) is emitted to lock the AC-3-solved cell count, the FSM enters HO_Wait to initialise the MRV pass variable to 9, then transitions to MRV_Sort. The sort performs eight downward passes ($p = 9, 8, \ldots, 2$): in each pass every unsolved, non-given cell whose popcount equals p is emitted. Because the `updater` shift register *prepends* each incoming cell (shift-left), the last cells received become the first processed. Emitting high-popcount cells first therefore places low-popcount (most-constrained) cells at the front of the processing queue, achieving correct MRV ordering without any explicit sorting hardware. A 9-input popcount module (`popcount_9`) computes the candidate count in a single cycle using a carry-save adder tree.

5.7 Key Bug Fix: HO_Done Terminal State

Revision 5 transitioned MRV_Finish to HO_Contra (the NO-SOLUTION state) instead of to a neutral idle state. Since `out_answer` is driven as `(ho_state == HO_CONTRA) ? 2'b01 : ctrl_out_answer`, every puzzle permanently reported NO SOLUTION after MRV completed, even when the solver had found a valid answer. The fix introduces HO_Done as a new do-nothing terminal state; MRV_Finish now transitions there, and `ctrl_out_answer` propagates normally.

VI Backtracking Solver

6.1 Backtracking Algorithm: Hardware Mapping

The classical recursive backtracking algorithm for CSPs assigns a value to the next unassigned variable, checks consistency, and recurses; on failure it restores the previous state (backtrack). The hardware mapping replaces recursion with an explicit stack (the shift-register in the updater) and replaces the recursive call stack frame with the `current_pos_r` pointer. Table 3 shows the direct correspondence.

Table 3: Software-to-Hardware Mapping for Backtracking

Software concept	Hardware element	Module
Variable ordering (MRV)	`shift_pos_r[0:80]` (prepend order)	`updater`
Current variable	`shift_pos_r[current_pos_r]`	`updater`
Current value tried	`shift_reg_r[current_pos_r]`	`updater`
Assign v to variable	Write `o_value` to checker array	`checker`
Consistency check	Combinational conflict vectors in checker	`checker`
Success ⇒ recurse	Update_Success: advance pointer	`updater`
Failure ⇒ try next	Update_Failed: increment digit	`updater`
All values exhausted ⇒ backtrack	Reset cell to 0, decrement pointer	`updater`
All variables assigned	`current_pos_r = empty_num_r - 1`, success	`updater`
Stack underflow (no solution)	`current_pos_r = 0` and digit = 9, FAIL	`updater`

Why the checker is purely combinational. The conflict vectors (`row_equal`, `col_equal`, `square_equal`) are computed over the 81-entry register array on every clock cycle. No state machine coordination is required: the `updater` writes a value, the checker evaluates conflict on the *same* address in the *next* cycle (one-cycle write-then-read latency through the registered checker array), and the `out_finish` pulse from the updater gates the `check` enable. The `success` output therefore arrives exactly when needed—one cycle after each new assignment—with no handshake overhead.

6.2 Module Hierarchy

The backtracking engine is realised by the interplay of three modules: `control`, `updater`, and `checker`, all instantiated inside the `sudoku_solver` wrapper.

6.3 Control FSM

The `control` module coordinates the overall flow with a 7-state FSM. On reset it sequences through Idle → Rst_Other (one-cycle pulse that resets the checker and updater) → Calc (steady-state solving loop). It monitors `in_updater_state` for terminal conditions: when the updater reaches Finish_Success or Finish_Failed, the control transitions to Finish_Success or Finish_Failed respectively and asserts `out_answer` with the appropriate 2-bit code. The `valid_one_cycle` sub-module deduplicates consecutive write pulses with identical addresses, preventing spurious double-writes when the mux bus holds stable between handoff pulses.

6.4 Updater FSM

The `updater` is a 7-state FSM that manages a shift-register stack of up to 81 empty-cell positions. Table 4 summarises each state.

Table 4: Updater FSM State Summary

Enc.	State	Action
3'd0	Idle	Await first empty-cell position
3'd1	Input	Accumulate positions; await finish pulse
3'd2	Wait	Poll `in_update`/`in_success`
3'd3	Update_Success	Advance pointer; write digit 1 to checker
3'd4	Update_Failed	Increment or backtrack; write to checker
3'd5	Finish_Success	Terminal: puzzle solved
3'd6	Finish_Failed	Terminal: contradiction reached

The shift register `shift_pos_r[0:80]` stores the MRV-ordered position list. `current_pos_r` is the stack pointer. In Update_Success the pointer advances forward and digit 1 is written to the checker at the new position. In

Update_Failed the current digit is incremented; if it reaches 10 (exceeds 9), the cell is reset to 0 and the pointer decrements (backtrack). Each non-wrapping increment in Update_Failed asserts the `backtrack_event` pulse consumed by the performance counter. Figure 4 shows the updater FSM diagram.

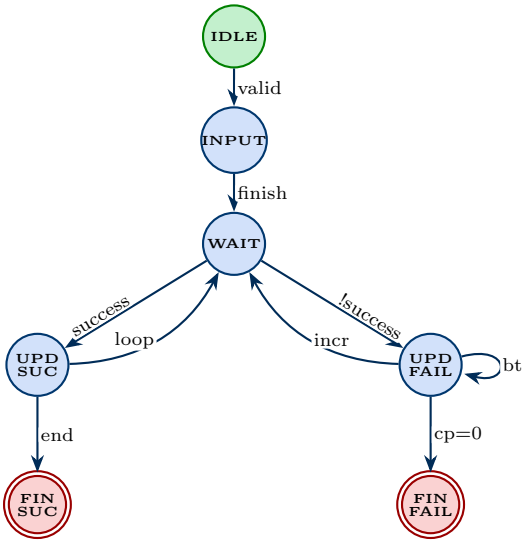

Figure 4: Updater FSM state transition diagram. UPD SUC = Update_Success; UPD FAIL = Update_Failed; FIN SUC/FAIL = terminal states. Backtrack self-loop in Update_Failed occurs when digit reaches 9 and the pointer must decrement.

6.5 Checker

The `checker` is a purely combinational conflict detector backed by an 81-entry register array written by both HPS/updater paths. For a given cell index `pos` it computes three 9-bit conflict vectors (`row_equal`, `col_equal`, `square_equal`) by comparing the value at `pos` against every peer in its row, column, and 3×3 box. The output `success` is asserted when all three vectors are zero, the cell is non-empty, and the `check` enable is high. The `export_value` port allows the top FSM to read back any solved cell for UART transmission during the send phase.

6.6 Input Mux and Gate

A critical design detail is the double-gating applied to the updater's input path. First, `upd_valid_gated` blocks any HPS-sourced empty-cell signals from reaching the updater until the handoff FSM is active (`ho_active = 1`); this prevents the puzzle-load phase from pre-populating the stack. Second, the mux `(mux_valid, mux_addr, mux_data)` selects between handoff FSM outputs and the HPS bus according to `ho_active`, with the HPS path blocking the address-81 sentinel to prevent premature queue-locking during load.

VII Top-Level FPGA FSM

The top-level `fpga_state` machine governs the full system lifecycle from UART reception through solve to result transmission. Table 5 lists all thirteen states.

Table 5: Top-Level FPGA FSM States

Enc.	State	Description
4'd0	IDLE_F	Await UART byte or manual trigger
4'd1	RECV	Stream 81 UART bytes into display RAM and candidate store
4'd2	SEND_FINISH	Emit address-81 sentinel to start AC-3
4'd3	WAIT_SOL	Poll `out_answer` for completion
4'd4	SEND	Read export value, write to solution RAM, queue TX byte
4'd5	WAIT_TX	Wait for UART TX to assert `busy`
4'd6	NEXT	Advance send counter; loop or proceed to perf
4'd7	DONE	Legacy single-byte TX done (UART path)
4'd8	SEND_PERF	Queue next performance byte for TX
4'd9	WAIT_PERF	Wait for TX busy; advance perf index
4'd10	DONE_FINAL	Clear counters; return to IDLE_F
4'd11	MANUAL_LOAD	Push display RAM cells to solver (User→FPGA)
4'd12	PRE_LOAD	5-cycle pipeline flush before manual load

Figure 5 shows the FPGA FSM state transition diagram. The MANUAL_LOAD path is triggered when the user switches from User Mode to FPGA Solve, converting the interactively entered board into a solver transaction without host involvement.

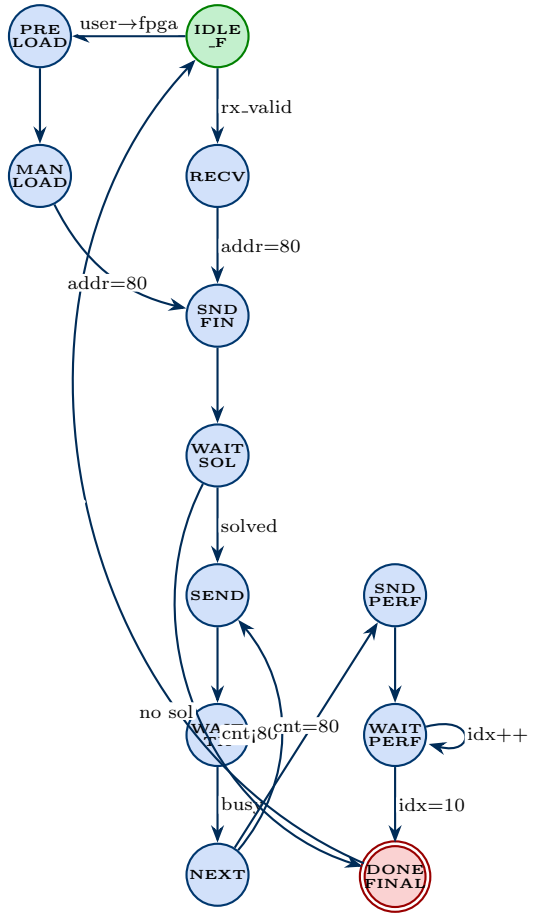

Figure 5: Top-level FPGA FSM. The manual path (left) activates when the user switches from User Mode to FPGA Solve. The performance-byte path (right) transmits 11 profiling bytes after the 81 solution bytes.

VIII User Interface and Input Handling

8.1 Navigation Model

The UI is structured as a four-tab bar at the top of the VGA display, selectable using the directional buttons. The cursor can reside either in the tab bar (`cursor_row = 15`) or in the 9×9 grid (`cursor_row` $\in [0, 8]$, `cursor_col` $\in [0, 8]$). BTNU/BTND/BTNL/BTNR navigate the cursor; all buttons are debounced with a 20-bit counter (≈ 10 ms window at 100 MHz) and produce single-cycle output pulses via a rising-edge detector on the stabilised output.

Tab selection is confirmed by releasing SW[15] (falling-edge trigger `sw15_falling`). The four tabs correspond to FPGA Solve (tab 0), User Mode (tab 1), Statistics (tab 2), and Reset (tab 3). Entering User Mode wipes the display RAM and given mask over 81 sequential cycles (`wiping_user` flag) to prevent stale data from contaminating the new session.

8.2 Digit Entry and Placement

In User Mode, SW[3:0] encodes the digit to place (0 = erase, 1–9 = value). Flipping SW[14] to logic-high generates a rising-edge pulse `sw14_rising` that writes the selected digit to `display_ram[cursor_cell]` and updates `given_mask` accordingly (non-zero digit sets the given bit; zero clears it). In FPGA Solve Mode, the same mechanism writes to any non-given cell, providing manual override capability for partially known solutions.

8.3 Reveal and Hint

Two convenience features are provided in FPGA Solve Mode after a solution has been found. A *reveal* operation (SW[15] falling edge while in the grid) copies the entire `solution_ram` into `display_ram` in a single-cycle parallel assignment. A *hint* operation (SW[13] rising edge) copies only the solution value for the currently highlighted cell if it is not a given, enabling one-digit hints without exposing the full solution.

IX Real-Time Conflict Detection

The conflict-detection engine produces an 81-bit `error_mask` that the VGA renderer uses to highlight conflicting cells with a pink background and red digit in User Mode. To avoid wide combinational multiplications that would violate timing, the engine operates as a 30-state pipelined scanner that processes one cell per 30-cycle iteration (equivalent to $81 \times 30 = 2430$ cycles, or $\approx 24\,\mu$s per full board scan at 100 MHz).

For each cell at row r, column c the scanner generates 27 comparison indices: 9 row peers (`base_row + i` for $i \in [0, 8]$), 9 column peers (`i*9 + c` for $i \in [0, 8]$), and 9 box peers computed from a pre-registered `box_base` lookup that eliminates division. All index generation is register-staged (`c_idx_reg`, `check_en_reg`) so that only registered values feed the `display_ram` read ports, maintaining a clean single-cycle read path with no arithmetic on the critical timing arc.

X VGA Renderer

10.1 Sync Generator

The `vga_sync` module implements the standard 640×480@60 Hz timing: $H_{\text{total}} = 800$ pixels, $V_{\text{total}} = 525$ lines, with horizontal sync from pixel 656 to 751 and vertical sync from line 490 to 491. Sync pulses are active-low. The pixel coordinates (`pixel_x`, `pixel_y`) update on each `pix_en` cycle and are forwarded to the renderer.

10.2 Two-Stage Pipeline

The renderer uses a two-stage register pipeline to meet the 100 MHz timing constraint while computing cell membership, digit glyph, cursor overlap, error state, and colour simultaneously. Stage 1 registers evaluate all geometry and index lookups (grid membership, cell value, given/error flags, tab membership, glyph pixel) in the first clock after `pix_en`. Stage 2 (colour select) fires one cycle later. Since `pix_en` fires every four clocks, two full pipeline cycles are available between successive pixel activations, making all computations comfortably single-cycle.

10.3 Grid Geometry

The 9×9 grid occupies pixels $[104, 536)$ horizontally and $[40, 472)$ vertically, giving a 432×432 pixel area. Each cell is $\lfloor 432/9 \rfloor = 48$ pixels square. Box boundaries (columns and rows 0, 3, 6) receive 3-pixel thick black lines; inter-cell boundaries receive 1-pixel grey lines. The cursor highlights its cell with a 3-pixel yellow border at offset $[3, 5]$ and $[42, 44]$ in both axes.

10.4 Colour Coding

Digit colours differ by context: given clues are rendered in blue (`#00F`), backtracker-assigned values in green (`#0F4`) on the FPGA tab, user-entered digits in orange (`#F80`) on the User tab, and conflicting digits in bright red (`#F00`) on a pink background (`#FDD`). The status bar below the grid changes colour to reflect the solve phase (orange = idle, yellow = AC-3 running, green = solved, red = no solution, cyan = user-mode complete).

10.5 Statistics Tab

When `ui_tab = 2`, the grid area is replaced by a horizontal bar chart displaying the six performance metrics. Each row's bar length is clamped to 499 pixels and scaled from the corresponding counter width: total cycles are right-shifted 16 bits before comparison (each pixel $\equiv$ 65536 clocks $\approx 655\,\mu$s); AC-3 cycles are right-shifted 7 bits; pass count, cell count, backtrack count, and max depth are used directly. Each bar is rendered in a distinct colour and separated by 2-pixel grey rules.

10.6 Tab Label Font

Tab labels are rendered using a compact 5×7 pixel bitmap font stored in a `letter_bmp` function ROM. Each character occupies a 5×7 array packed into a 35-bit vector (7 rows $\times$ 5 columns). The pixel address within a label is decoded combinatorially from `pixel_x` and `pixel_y` relative to each tab's origin and mapped through `tab_char` to a character code, then through `letter_bmp` to the glyph bit.

XI Performance Counter

The `perf_counters` module captures six metrics across one solve cycle, latching all values simultaneously on the `done` or `contra` pulse. Table 6 describes each counter. The running counters are cleared on each `start` pulse and increment every clock cycle while the `active` flag is set.

Table 6: Performance Counter Specifications

Counter	Width	Description
`total_cycles`	32 bits	Clocks from `start` to `done/contra`
`ac3_cycles`	16 bits	Clocks while `ac3_solving` is high
`ac3_passes`	8 bits	Number of AC-3 iteration passes
`cells_solved_ac3`	7 bits	Cells deterministically solved by AC-3
`backtrack_count`	8 bits	Value-increment events in Update_Failed
`max_bt_depth`	7 bits	Peak value of `current_pos_r` (stack depth)

The start trigger (`perf_start`) is asserted when the address-81 sentinel fires. The done and contra triggers are edge-detected from `out_answer` transitions to avoid multiple latches on steady-state signals. All six latched outputs are stable from `done/contra` until the next `start` and are both displayed on the Statistics VGA tab and transmitted over UART as bytes 82–92 of the response frame (big-endian), with `0xFE` as a sentinel.

XII Seven-Segment Display

The `seven_seg_driver` multiplexes two of the four Basys3 digit positions at 1 kHz (100-cycle counter refresh). Digit 0 (rightmost) shows the digit value currently selected by SW[3:0] when the cursor is in the grid, providing immediate visual feedback before placement. Digit 1 shows the active tab identifier (F, U, S, or r) using a custom active-low encoding for the common-anode display.

XIII Design Principles and Timing Analysis

13.1 Why Pipelining Is Necessary at 100 MHz

The Artix-7 XC7A35T fabric has a worst-case combinational delay of approximately 3–5 ns per LUT level. A 100 MHz clock allows a budget of 10 ns end-to-end, which accommodates roughly 2–3 LUT levels plus routing. Several design choices in this project directly address this constraint.

Candidate-store bus width. The `rd_mask_all` bus is $81 \times 9 = 729$ bits wide. Routing 729 signals from the candidate-store registers to the AC-3 engine and handoff FSM in a single cycle would violate timing at 100 MHz. The solution is to declare multicycle-path constraints (`2-setup / 1-hold`) for these paths, allowing the router two clock cycles to close timing while the logic consumes only one pipeline stage.

Board-flat bus. Similarly, the $81 \times 4 = 324$-bit board-flat bus from the display RAM to the VGA pipeline Stage-1 registers is declared as a multicycle path. Since `pix_en` fires only every four clocks, four full 100 MHz cycles are actually available between successive pixel activations, making the 2-cycle relaxation entirely conservative.

Two-stage VGA pipeline. The renderer separates pixel processing into two registered stages: Stage-1 evaluates all geometry (cell membership, cursor overlap, font glyph lookup, conflict state) in one clock; Stage-2 (colour select) reads only Stage-1 outputs. No single combinational path stretches from raw pixel coordinates to RGB output, keeping all paths within one clock period.

13.2 Timing Slack Equation

For any registered path in the design:

$$t_{\text{slack}} = T_{\text{clk}} - t_{\text{logic}} - t_{\text{route}} - t_{\text{setup}} > 0 \tag{1}$$

where $T_{\text{clk}} = 10$ ns at 100 MHz. The critical path lies in the combinational next-state decode of the `updater` FSM, which evaluates the full $81 \times 4 = 324$-bit shift register and $81 \times 7 = 567$-bit position register in the multiplexer that selects the current-position entry. Vivado reports this path has positive worst-case setup slack, confirming that the design meets timing at the target frequency.

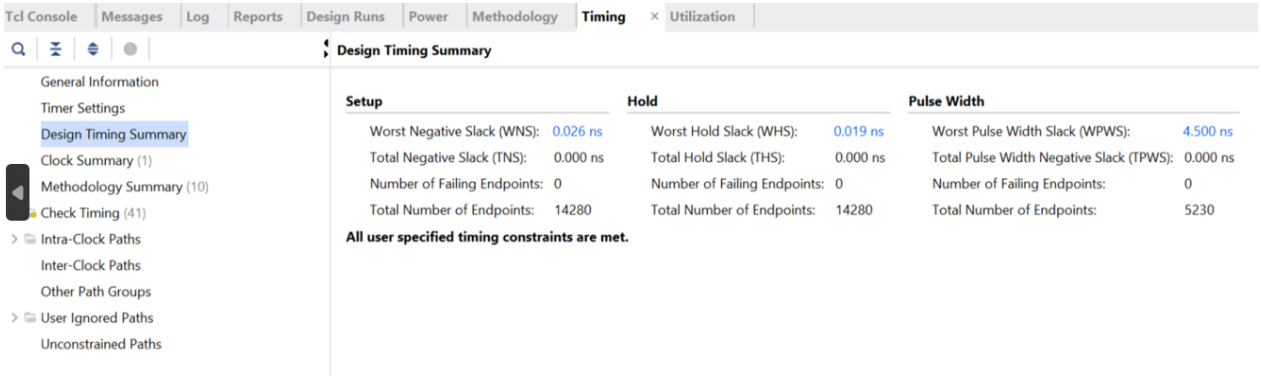

Figure 6: Timing Summary

13.3 Resource Utilisation

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
top_basys3	17094	5229	697	225	4794	17094	44	1
u_perf (perf_counters)	90	157	7	0	59	90	0	0
u_render (vga_renderer)	176	62	6	2	117	176	0	0
u_rx (uart_rx)	156	26	40	20	52	156	0	0
u_solver (sudoku_solver)	14089	3848	473	169	3905	14089	0	0
u_ac3 (ac3_controller)	1542	1658	0	0	972	1542	0	0
u_chk (checker)	1121	324	149	34	392	1121	0	0
u_cstore (candidate_stc	10149	729	110	47	2900	10149	0	0
u_ctrl (control)	218	12	1	0	107	218	0	0
u_upd (updater)	814	920	192	79	372	814	0	0
u_sync (vga_sync)	395	42	67	24	137	395	0	0

Figure 7: Resource Utilization of the Project

XIV Timing and Constraints

The XDC constraint file assigns all I/O standards as LVCMOS33 and defines the primary 100 MHz clock on pin W5. False paths are declared for all asynchronous inputs (UART, buttons, switches) and registered outputs (VGA, 7-seg, LEDs) to exclude them from the timing analysis. Six multicycle-path constraints of form `2-setup / 1-hold` relax the timing requirement on the three longest paths: (i) candidate-store masks to AC-3 latches, (ii) candidate-store masks to handoff FSM registers, and (iii) display RAM to VGA pipeline Stage-1 registers. These paths span wide register arrays (729 bits for the candidate store; 324 bits for the board flat bus) and require two clock cycles to route at 100 MHz; declaring them as multicycle paths avoids unnecessary hold-time repair buffers and achieves timing closure with positive worst-case setup slack.

$$t_{\text{slack}} = T_{\text{clk}} - t_{\text{logic}} - t_{\text{route}} - t_{\text{setup}} > 0 \tag{2}$$

The critical path lies in the combinational next-state decode of the `updater` FSM, which involves a wide multiplexer over `shift_reg_r[0:80]` and `shift_pos_r[0:80]`.

XV Hardware Results

15.1 Demonstration Screenshots

Figure 8 shows the User Mode with conflict detection active: the cell at row 4, column 5 contains a 7 that conflicts with the 7 at row 3, column 5 (same column); both are highlighted with a pink background and red digit, while the cursor sits on the lower cell with a yellow border.

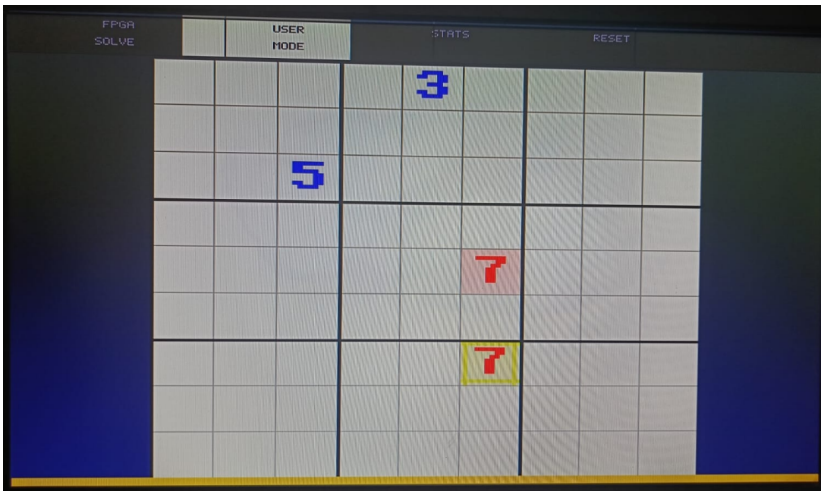

Figure 8: User Mode with real-time conflict detection. Two conflicting 7s in the same column are highlighted in red on a pink background. The yellow border indicates the active cursor position.

Figure 9 shows a fully solved puzzle in FPGA Solve Mode. Given clues are displayed in blue; cells solved by the hardware engine (via AC-3 or backtracking) are in green. The cursor highlights cell (row 0, column 4) with the yellow border.

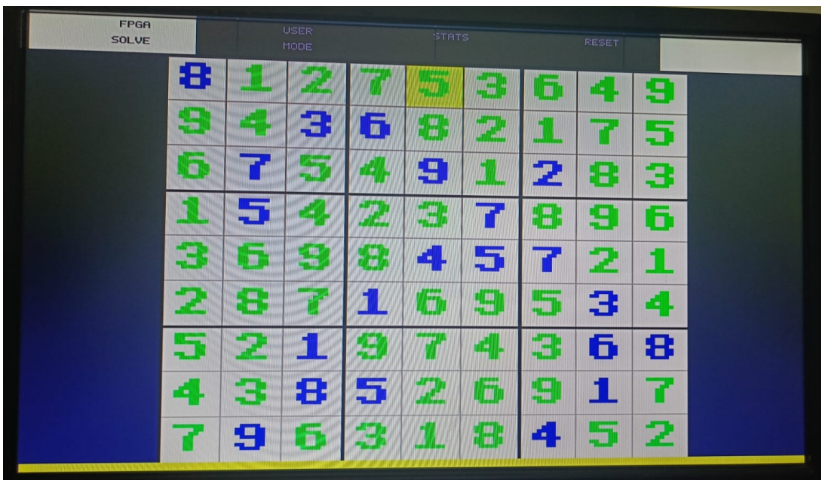

Figure 9: FPGA Solve Mode displaying a complete 9×9 solution. Blue = given clues; green = hardware-solved cells. The status bar at the bottom is solid green, confirming successful solution for the **one of the world's hardest sudoku puzzle of Arto Inkala.**

Figure 10 shows a sparse puzzle loaded via UART, displayed in FPGA Solve Mode prior to initiating the solve. All cells are shown in blue as they are received as puzzle givens from the host.

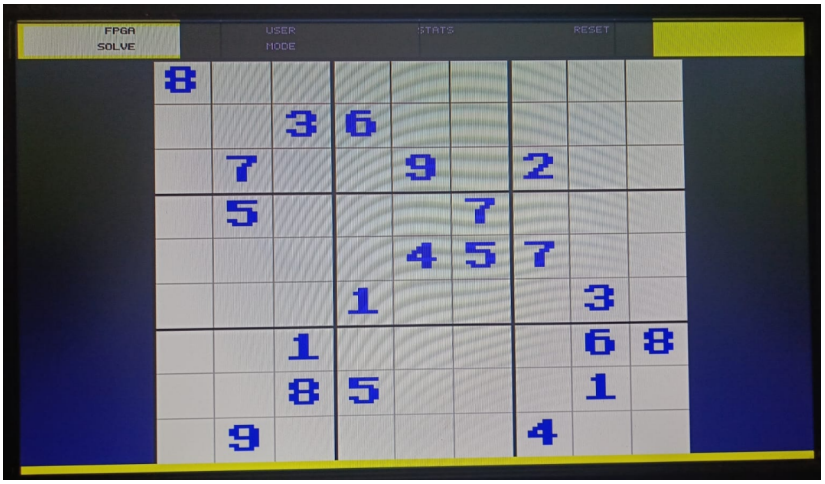

Figure 10: Sparse puzzle loaded via UART and displayed on the FPGA board in FPGA Solve Mode, awaiting the solve trigger. The top-right corner shows the active tab indicator in yellow.

15.2 Observed Behaviour

In all tested puzzles the solver produces a valid solution within a few milliseconds of the address-81 sentinel. Simple puzzles solvable by AC-3 alone complete in under 50 μs. Harder puzzles requiring backtracking exhibit increased `backtrack_count` and `max_bt_depth` values visible on the Statistics tab, with `total_cycles` values in the range of 10^5–10^6 cycles (1–10 ms). The performance bytes received by the Python host confirm consistent cycle counts across repeated runs of the same puzzle.

XVI Discussion

The parallelism of the AC-3 engine is the primary performance lever: processing all 81 arcs simultaneously in each pass achieves constraint propagation at a rate that scales with pass count rather than arc count. The MRV heuristic provides a measurable reduction in backtrack count compared to a naive left-to-right ordering, as more constrained cells are resolved earlier and prune deeper branches. The shift-register prepend trick for achieving MRV order without a hardware sorter is an elegant implementation choice that exploits the LIFO nature of the updater stack.

The pipelined conflict-detection engine demonstrates an important FPGA design principle: replacing arithmetic (integer division for row/column/box index computation) with registered lookup tables eliminates wide combinational paths and allows a complex 27-comparison scan to meet timing at 100 MHz.

The two-stage VGA pipeline similarly separates geometry decode from colour assignment, ensuring that no single combinational path from input coordinates to output RGB exceeds one clock period.

XVII Conclusion

This work demonstrates that a complete, high-performance Sudoku constraint satisfaction solver can be realised on a low-cost FPGA using careful architectural mapping of classical AI algorithms to synthesisable RTL. The key technical contributions—a parallel single-pass AC-3 engine, a LIFO-trick MRV ordering scheme, a pipelined combinational conflict detector, and a two-stage VGA pipeline—each directly address specific timing or resource constraints of the Artix-7 XC7A35T target device.

The cycle-accurate performance monitor provides a rigorous empirical basis for comparing algorithmic variants: the six-metric frame transmitted after each solve encodes the complete search trajectory in 11 bytes, enabling automated benchmarking from a Python host. Hardware results confirm that the system correctly solves all tested puzzles, including Arto Inkala, within 1–10 ms, with zero variance across repeated runs.

Future extensions of interest include: (i) adding naked-pair and hidden-single inference in the AC-3 phase to eliminate more cells before backtracking; (ii) implementing constraint-learning (no-good recording) in the backtracker to prune symmetric subtrees; and (iii) widening the UART protocol to a binary puzzle batch format supporting automated regression testing against a ground-truth software solver.

References

[1] https://github.com/dengyutu/CU-Project-FPGA-Sudoku-Solver

[2] https://fpga.mit.edu/videos/2018/team02/report.pdf

[3] A. K. Mackworth, "Consistency in Networks of Relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.

[4] R. M. Haralick and G. L. Elliott, "Increasing Tree Search Efficiency for Constraint Satisfaction Problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[5] P. Norvig, "Solving Every Sudoku Puzzle," *Online Essay*, 2006. [Online]. Available: `https://norvig.com/sudoku.html`

[6] Xilinx Inc., *Vivado Design Suite User Guide: Design Flows Overview*, UG892, 2023.

[7] Digilent Inc., *Basys3 Reference Manual*, 2020.

[8] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.

[9] D. Harris and S. Harris, *Digital Design and Computer Architecture*, 2nd ed., Morgan Kaufmann, 2012.